

FACULTÉ DES SCIENCES BEN M'SICK

DÉPARTEMENT MATHÉMATIQUES & INFORMATIQUE

MASTER D'EXCELLENCE INTELLIGENCE ARTIFICIELLE

Module : MLOps

Analyse des parcours d'apprentissage avec une chaîne MLOps

Sujet : EduCluster - Segmentation des profils étudiants

Réalisé par :

Mohammed Hmimid
Chakir Abderrahmane
Bahri Ilhame
Aymene Chouki
Aldiebes Ghanem Israa
Khadija Zaafa
Othman Salahi
Makrani Mohamed

Encadré par :

M. Mohammed AITDAOUD
M. Taoufik Amzil

Remerciements

Nous adressons nos sincères remerciements à **M. Mohammed AITDAOUD**, encadrant de ce projet, pour son accompagnement, ses orientations et les échanges qui ont guidé la réalisation de cette chaîne MLOps. Nous remercions également l'équipe pédagogique du Master d'Excellence Intelligence Artificielle pour le cadre scientifique et technique offert tout au long du module.

Résumé

EduCluster est une plateforme d'analyse des parcours d'apprentissage fondée sur le jeu de données OULAD (*Open University Learning Analytics Dataset*). Elle regroupe les étudiants selon leurs comportements d'apprentissage en ligne grâce à un modèle K-Means non supervisé. Le projet couvre l'ensemble du cycle MLOps : versionnement des données avec DVC, ingestion avec dlt, stockage analytique dans DuckDB, transformation avec dbt, préparation de 34 caractéristiques, entraînement et évaluation, suivi avec MLflow, orchestration avec Dagster, exposition par FastAPI, visualisation dans une interface React, conteneurisation Docker et automatisation CI/CD avec GitHub Actions et Komodo.

Les résultats de référence portent sur 32 593 observations. Le modèle distingue deux profils : un segment actif et engagé (78,6 %) et un segment à faible engagement nécessitant potentiellement un accompagnement (21,4 %). Cette segmentation ne constitue pas une prédiction certaine de réussite ou d'échec ; elle fournit un outil d'aide à l'analyse des comportements d'apprentissage.

Mots-clés : MLOps, learning analytics, OULAD, clustering, K-Means, DVC, dbt, MLflow, FastAPI, Docker, CI/CD.

Table des matières

Introduction générale	1
Problématique	1
Objectifs du projet	1
Périmètre et principe éthique	1
Positionnement dans l'analyse des parcours d'apprentissage	2
Démarche de réalisation et niveau de preuve	2
Lecture attendue des contributions	3
I - Présentation de l'application et architecture	4
1. Présentation d'EduCluster	4
2. Architecture globale	4
3. Organisation du dépôt	5
4. Besoins fonctionnels et parcours des utilisateurs	5
5. Séparation entre calcul différé et prédiction à la demande	6
6. Responsabilités et échanges entre composants	6
7. Exigences de qualité et critères d'acceptation	7
II - Pipeline de données	8
1. Jeu de données OULAD	8
2. Versionnement avec DVC	8
3. Ingestion avec dlt et stockage DuckDB	8
4. Transformation et validation avec dbt	9
5. Vue d'ensemble du flux de données	9
6. Granularité des observations et clés de rapprochement	10
7. Différence entre stockage distant, cache et base analytique	10
8. Lecture des agrégations d'engagement	11
9. Évaluations, retards et données manquantes	11
III - Préparation, entraînement et évaluation	13
1. Construction des caractéristiques	13
2. Prétraitement	13
3. Comparaison des algorithmes	13
4. Entraînement du modèle final	14
5. Évaluation et résultats	14
6. Artefacts produits	15
7. Effet des transformations sur les distances	15

8. Interprétation géométrique et choix de deux groupes	16
9. Comparaison expérimentale et limites des preuves disponibles	17
10. Lecture critique des métriques de référence	17
11. Temporalité, fuites d'information et validation future	18
IV - Traçabilité et orchestration MLOps	19
1. Suivi des expériences avec MLflow	19
2. Reproductibilité avec DVC	19
3. Orchestration avec Dagster	19
4. Cycle de vie du modèle	20
5. Versionnement du modèle et contrat des artefacts	20
6. Journalisation du modèle existant dans MLflow	20
7. Matérialisation, tests et métadonnées Dagster	21
8. Séparation du serveur et du worker	21
9. Publication contrôlée du modèle	21
V - API, dashboard et conteneurisation	22
1. API FastAPI	22
1.1. Séquence d'une prédiction	22
2. Dashboard React	22
3. Conteneurisation Docker	23
4. Contrat des requêtes et validation des valeurs	23
5. Provenance des statistiques et comportement en cas d'absence	24
6. Communication et configuration des interfaces	24
7. Persistance et cycle de chargement	25
VI - Intégration et déploiement continu	26
1. Architecture CI/CD	26
2. Validation de l'API et du modèle persisté	26
3. Validation du frontend	26
4. Déploiement Komodo	26
5. Garanties apportées	27
6. Les quatre contrôles avant livraison	27
7. Vérification après déploiement	27
VII - Reproduction du projet	28
1. Prérequis	28
2. Installation locale	28
3. Reproduction complète du pipeline de données et ML	28
4. Lancement local des services	28

5. Lancement simplifié avec Docker	29
6. Organisation des données et des chemins	29
7. Exécution depuis Dagster	29
VIII - Validation, limites et perspectives	30
1. Validation du fonctionnement	30
2. Limites actuelles	30
3. Perspectives d'amélioration	30
4. Apports pédagogiques du projet	30
6. Interprétation pédagogique et limites des données	31
7. Surveillance future et évolution des distributions	31
8. Perspectives de consolidation	32
Conclusion générale	33
Bilan méthodologique et perspectives de transfert	33

Introduction générale

Les environnements numériques d'apprentissage produisent une grande quantité de traces : consultations de contenus, participation aux forums, utilisation des quiz, fréquence de connexion, remise des évaluations et évolution de l'engagement dans le temps. Exploitées avec méthode, ces traces permettent de mieux comprendre les parcours des apprenants.

L'objectif d'EduCluster est de transformer ces données brutes en profils comportementaux interprétables, puis de rendre le résultat reproductible et exploitable. Le projet ne se limite donc pas à entraîner un algorithme. Il met en place une chaîne complète qui relie la donnée, le modèle, l'API, l'interface et le déploiement automatisé.

Problématique

Comment construire une solution reproductible permettant de segmenter des étudiants selon leurs habitudes d'apprentissage en ligne, de contrôler la qualité du modèle, de servir de nouvelles prédictions et de déployer l'ensemble de manière fiable ?

Objectifs du projet

- centraliser et transformer les données OULAD dans une base analytique locale ;
- construire des variables représentatives de l'engagement et des évaluations ;
- comparer plusieurs méthodes de clustering et retenir une solution opérationnelle ;
- garantir la traçabilité des données, du modèle et des expériences ;
- exposer les résultats dans une API et un tableau de bord accessibles ;
- automatiser les tests, la construction des images et le déploiement.

Périmètre et principe éthique

Le clustering s'appuie sur 34 variables comportementales. Les variables démographiques ainsi que le résultat final de l'étudiant ne sont pas utilisées pour créer les groupes. Le terme « à risque » désigne uniquement un profil de faible engagement observé ; il ne doit pas être interprété comme une décision automatique ou une certitude d'échec.

Positionnement dans l'analyse des parcours d'apprentissage

L'analyse des parcours d'apprentissage consiste à étudier les traces laissées par les apprenants afin de mieux comprendre leurs pratiques, leur progression et leurs difficultés éventuelles. Dans EduCluster, cette analyse prend la forme d'une segmentation de comportements observés sur une plateforme pédagogique. Les interactions et les évaluations sont résumées dans une représentation numérique, puis rapprochées selon leur similarité. Le projet s'inscrit donc dans ce domaine, avec un périmètre précis : il décrit des profils d'engagement à partir d'indicateurs agrégés. Il ne reconstitue pas encore une succession détaillée d'états pédagogiques, ni un chemin personnalisé entre des compétences à acquérir.

Cette précision est essentielle pour comprendre la portée du travail. Un étudiant qui consulte régulièrement des ressources, participe aux activités et remet ses évaluations présente un comportement différent d'un étudiant dont les traces sont rares. Toutefois, les données ne permettent pas à elles seules d'expliquer ces différences. Elles ne décrivent pas le travail réalisé hors de la plateforme, les conditions matérielles d'étude ou les échanges informels. Le modèle rend visibles des régularités dans les observations disponibles. L'interprétation de ces régularités reste une activité pédagogique qui doit tenir compte du contexte et de la parole de l'apprenant.

Le choix d'une approche non supervisée répond à une intention exploratoire. Les groupes ne sont pas définis à l'avance par une catégorie de réussite ou d'échec ; ils émergent d'une comparaison entre observations. Cette démarche offre une première organisation d'une population hétérogène et peut servir de point de départ à une analyse plus fine. Elle permet aussi d'aborder, dans le cadre du module MLOps, les difficultés de passage d'une expérience scientifique à une application utilisable. La fiabilité de cette application dépend autant de la préparation des données et de la cohérence des versions que de l'algorithme lui-même.

Démarche de réalisation et niveau de preuve

La rédaction de ce rapport s'appuie sur les scripts Python, les modèles SQL, les configurations et les tests présents dans le projet. Une distinction est maintenue entre le comportement décrit par le code, les résultats de référence livrés dans les fichiers et les améliorations proposées. L'existence d'un workflow ne prouve pas qu'un déploiement distant a réussi ; de même, la présence d'une métrique dans un fichier ne constitue pas une nouvelle exécution de l'entraînement. Les valeurs présentées sont celles des résultats de référence du projet, et aucune performance supplémentaire n'est inventée pour compléter la présentation.

Cette méthode de lecture est particulièrement adaptée à un rapport MLOps. Une chaîne industrialisée doit pouvoir être comprise à partir de ses entrées, de ses sorties et de ses dépendances. Chaque composant est donc étudié selon les données qu'il reçoit, le traitement qu'il applique, les artefacts qu'il produit et les conditions nécessaires à sa reproduction. Les difficultés concrètes de configuration sont également décrites lorsqu'elles apparaissent dans le dépôt. Cette approche rend le document utile à la fois pour présenter les choix de conception et pour accompagner une reprise du travail par une autre équipe.

Le projet étudié est l'application EduCluster située à la racine de l'espace de travail. Le sous-dossier projet-v1 correspond à une autre version et ne sert pas de base à la description

fonctionnelle actuelle. Cette séparation évite de confondre des technologies, des objectifs ou des instructions de lancement provenant de versions différentes. Le rapport présente d'abord l'architecture, puis le traitement des données, la modélisation, la traçabilité et la mise à disposition. La reproduction et les perspectives terminent le parcours afin de relier les concepts expliqués à des opérations vérifiables.

Lecture attendue des contributions

Les contributions peuvent être appréciées selon trois questions complémentaires. La première concerne la compréhension des données : les indicateurs décrivent-ils de façon cohérente les traces disponibles ? La deuxième concerne le calcul : une autre personne peut-elle reconstruire les mêmes entrées et retrouver un modèle compatible ? La troisième concerne l'usage : l'utilisateur sait-il ce que représentent les résultats et dans quelles conditions ils ont été produits ? Ces questions structurent l'évaluation du projet au-delà de la simple présence d'un algorithme ou d'une interface. Elles relient les objectifs de formation du module aux exigences pratiques rencontrées pendant la réalisation.

Le rapport doit également permettre une présentation orale progressive. Une démonstration peut commencer par l'objectif pédagogique et la page de synthèse, puis remonter vers l'API, les artefacts du modèle et les transformations de données. Le lecteur technique peut suivre le chemin inverse et examiner les dépendances depuis OULAD. Ces deux lectures aboutissent au même constat : une réponse affichée dans le navigateur dépend d'une succession de choix explicites. La valeur du travail collectif réside dans la capacité à rendre cette succession compréhensible, reproductible et modifiable sans perdre le lien entre les résultats et leur contexte.

I - Présentation de l'application et architecture

1. Présentation d'EduCluster

EduCluster est une application de *learning analytics* dédiée à l'analyse des parcours d'apprentissage. Elle fournit une vue synthétique de la population étudiée, affiche les deux segments identifiés par le modèle et permet de positionner un nouveau profil à partir de ses indicateurs d'activité.

Composant	Rôle	Technologies
Données	Versionner, charger et transformer les sources OULAD	DVC, Google Drive, dlt, DuckDB, dbt
Machine Learning	Préparer les variables, entraîner et évaluer le clustering	Pandas, NumPy, scikit-learn
Traçabilité	Suivre paramètres, métriques et artefacts	MLflow, DVC
Service	Servir les statistiques et les prédictions	FastAPI, Uvicorn
Interface	Présenter les profils et collecter de nouvelles valeurs	React, Vite, Recharts
Industrialisation	Orchestrer, tester, conteneuriser et déployer	Dagster, Docker, GitHub Actions, Komodo

Tableau 1 - Composants principaux de la solution

2. Architecture globale

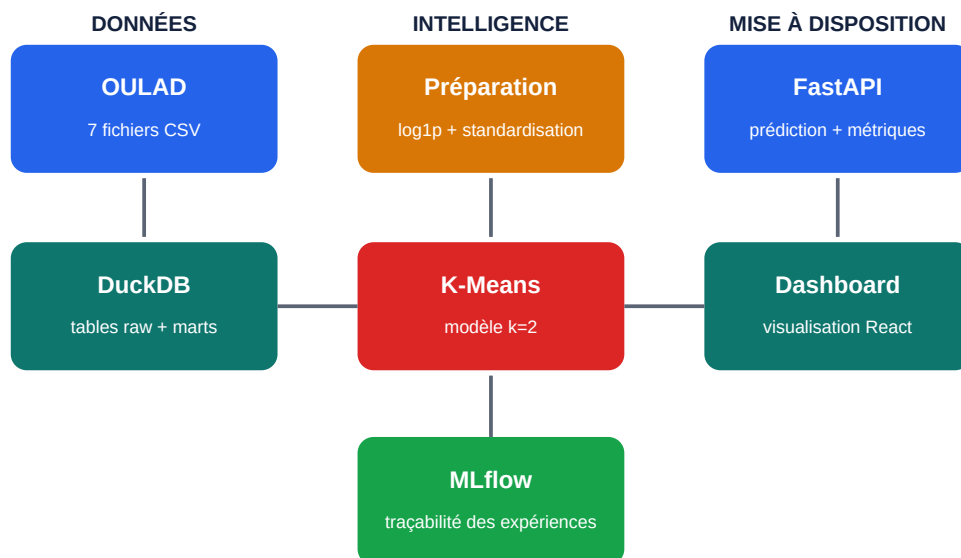


Figure 1 - Architecture fonctionnelle d'EduCluster

L'architecture sépare trois responsabilités. La couche données assure la disponibilité et la qualité des informations. La couche intelligence transforme les indicateurs et applique le clustering. La couche de mise à disposition rend les résultats accessibles aux utilisateurs. MLflow relie l'entraînement à la traçabilité expérimentale.

3. Organisation du dépôt

```
1 Mlops-Project-main/
2 |-- api/           # API FastAPI et données de présentation
3 |-- dashboard/     # Interface React/Vite
4 |-- data/raw/      # Pointeur DVC vers OULAD
5 |-- data/processed/ # Features, scaler, résultats du modèle
6 |-- dbt/educluster/ # Modèles SQL et tests de données
7 |-- dlt_pipeline/  # Ingestion des CSV dans DuckDB
8 |-- ml/           # Préparation, comparaison et entraînement
9 |-- models/        # Modèles K-Means sérialisés
10 |-- pipelines/     # Orchestration Dagster
11 |-- tests/         # Tests API, dashboard et déploiement
12 |-- docker-compose.yml # API, MLflow et dashboard
13 |-- dvc.yaml       # Étapes reproductibles du pipeline ML
14 `-- .github/workflows/ # Intégration et déploiement continus
```

Listing 1 - Structure principale du projet

4. Besoins fonctionnels et parcours des utilisateurs

Le premier besoin fonctionnel consiste à donner une vue compréhensible de la population analysée. Un utilisateur doit pouvoir consulter le nombre d'observations, connaître la proportion des deux groupes et comprendre leur interprétation générale. Cette vue répond à une question simple : comment se répartissent les comportements d'apprentissage dans les données disponibles ? L'interface doit également indiquer l'origine des chiffres affichés, car une statistique issue d'un entraînement local récent n'a pas la même signification qu'une valeur de référence fournie avec le projet. La lisibilité de cette provenance participe directement à la confiance dans le tableau de bord.

Le second besoin concerne l'analyse d'un nouveau profil. L'utilisateur renseigne les indicateurs d'activité et d'évaluation, puis soumet le formulaire à l'API. Le service transforme les valeurs, applique le modèle enregistré et renvoie une affectation. La réponse doit rester simple à lire, mais son utilisation suppose que les valeurs saisies respectent les mêmes définitions que celles de l'entraînement. Une moyenne calculée sur tous les jours du cours ne peut pas être substituée sans précaution à une moyenne calculée seulement sur les jours actifs. Le formulaire est donc une interface vers un contrat de données précis.

Un troisième public est constitué des personnes chargées de maintenir le système. Elles doivent pouvoir retrouver les scripts d'entraînement, comprendre les dépendances du pipeline, contrôler les métriques et diagnostiquer une panne. Le projet répond à ce besoin par l'organisation du code, les fichiers de configuration, les tests et les outils de suivi. Les responsabilités sont distribuées entre plusieurs composants spécialisés plutôt que concentrées dans un seul script. Cette organisation facilite une intervention ciblée : corriger une requête SQL, réentraîner le modèle ou modifier un composant React correspondent à des opérations distinctes dans le cycle de développement.

5. Séparation entre calcul différé et prédiction à la demande

La préparation des données et l'entraînement sont des traitements différés. Ils peuvent manipuler un volume important d'interactions, agréger les observations, recalculer les statistiques de normalisation et produire plusieurs fichiers. Leur durée ne doit pas conditionner chaque consultation du tableau de bord. Pour cette raison, l'application déployée utilise des artefacts déjà construits. L'API charge le modèle et le scaler au démarrage, puis applique ces objets aux requêtes reçues. La séparation entre construction et utilisation permet de répondre rapidement à une demande sans relancer l'ingestion ou l'optimisation de K-Means.

Cette distinction explique aussi pourquoi l'API n'a pas besoin d'interroger Google Drive pour chaque prédiction. Le stockage distant DVC intervient dans la récupération des données et dans la reproduction du pipeline. DuckDB intervient pendant la transformation analytique. Une fois le modèle construit, les calculs nécessaires à une nouvelle affectation reposent principalement sur les paramètres du scaler et les centres des clusters. L'interface Web dépend donc d'un service de prédiction opérationnel et de ses artefacts, tandis que le pipeline de formation dépend d'un ensemble plus large de fichiers, d'autorisations et de traitements.

La contrepartie de cette architecture est qu'une nouvelle donnée brute ne modifie pas automatiquement le comportement de l'application. Il faut décider d'un réentraînement, produire un ensemble cohérent d'artefacts, le vérifier et le livrer au service. Cette étape de publication constitue une frontière importante du cycle MLOps. Elle évite de modifier silencieusement les profils pendant l'utilisation, mais impose une politique claire de versionnement. Une évolution du modèle devrait être accompagnée d'une identification de la version et d'une vérification de ses conséquences sur les statistiques présentées aux utilisateurs.

6. Responsabilités et échanges entre composants

Les échanges entre composants sont relativement simples : fichiers CSV pour les données brutes et préparées, tables DuckDB pour l'analyse, fichiers Joblib pour les objets Python entraînés et messages JSON pour l'API. Ces formats jouent le rôle de contrats entre les étapes. Le pipeline de préparation produit une matrice dont les colonnes doivent correspondre à la liste attendue par l'entraînement ; le service de prédiction doit ensuite reconstruire exactement cet ordre. Le fait d'utiliser des outils différents ne dispense donc pas de définir explicitement ce qui circule entre eux et les propriétés attendues de chaque sortie.

Une responsabilité particulièrement importante concerne la cohérence du lot publié. Un modèle, un scaler et une liste de caractéristiques constituent un ensemble fonctionnel. Il serait incorrect de remplacer uniquement le scaler par celui d'un autre entraînement : les coordonnées des nouveaux profils seraient alors exprimées dans un espace différent de celui des centroïdes. Les métriques et les profils descriptifs devraient également provenir du même entraînement. Cette dépendance logique existe même si les fichiers sont stockés dans des répertoires séparés. Elle justifie une validation commune avant toute mise à jour du service.

La configuration joue enfin le rôle de lien entre les composants logiciels et leur environnement réel. Les chemins vers DuckDB, les ports publics, les origines autorisées et les URL utilisées par le navigateur doivent correspondre au mode d'exécution choisi. Le code

applicatif peut être correct alors qu'une mauvaise configuration empêche les échanges. C'est pourquoi le rapport distingue les paramètres du développement local de ceux de Docker Compose et décrit les vérifications à effectuer avant le lancement. Dans une chaîne MLOps, la configuration est une partie du système à versionner et à contrôler.

7. Exigences de qualité et critères d'acceptation

Les exigences de qualité traduisent les objectifs généraux en propriétés observables. La disponibilité implique que l'API démarre avec les artefacts attendus et que le navigateur puisse la joindre. La cohérence impose que les 34 variables respectent les mêmes définitions à l'entraînement et à la prédiction. La traçabilité demande qu'un résultat soit relié à des fichiers et à une configuration identifiables. Enfin, la lisibilité suppose que les deux profils soient décrits avec un vocabulaire compréhensible et que la provenance des statistiques soit affichée. Ces propriétés permettent de discuter la réussite du projet à partir d'éléments vérifiables.

Un critère d'acceptation technique pourrait être formulé ainsi : pour un profil de référence complet, l'appel HTTP doit renvoyer la même affectation que l'application directe du scaler et du modèle persistés. Un critère de présentation pourrait exiger que les effectifs affichés correspondent au total communiqué par l'API et que l'absence de données soit signalée. Un critère de reproduction pourrait demander qu'un environnement documenté construise le mart et les sorties attendues. Les tests existants couvrent déjà certains de ces aspects ; les autres donnent une direction concrète pour organiser les validations futures sans leur attribuer un statut de résultat acquis.

II - Pipeline de données

1. Jeu de données OULAD

OULAD est un jeu de données anonymisé issu d'un environnement universitaire à distance. Sept fichiers CSV décrivent les cours, les étudiants, les inscriptions, les évaluations et les interactions avec les ressources pédagogiques virtuelles.

Fichier	Contenu exploité
studentInfo.csv	Profil académique, crédits, tentatives et résultat final
studentRegistration.csv	Dates d'inscription et de désinscription
studentVle.csv	Interactions quotidiennes et nombre de clics
vle.csv	Type des ressources : contenu, forum, quiz, wiki, etc.
assessments.csv	Évaluations prévues, dates et pondérations
studentAssessment.csv	Soumissions, dates, scores et retard
courses.csv	Modules, présentations et durées

Tableau 2 - Sources composant le jeu de données

2. Versionnement avec DVC

Git reste réservé au code et aux petits fichiers. Les données brutes, plus volumineuses, sont suivies avec DVC. Le fichier `data/raw/oulad.dvc` contient l'empreinte de la version attendue, tandis que les données réelles sont stockées dans un espace Google Drive configuré comme dépôt distant DVC.

```
1 dvc remote list
2 dvc pull      # télécharge la version référencée des 7 fichiers OULAD
3 dvc push      # publie une nouvelle version dans le stockage distant
```

Listing 2 - Commandes principales de versionnement des données

Cette séparation garantit qu'un entraînement peut être associé à une version précise de la donnée sans alourdir le dépôt Git.

3. Ingestion avec dlt et stockage DuckDB

Le script dlt lit les sept fichiers CSV et crée des ressources remplaçables. À chaque exécution, les tables brutes sont chargées dans la base locale `oulad_pipeline.duckdb`, dans le schéma `raw`. DuckDB a été retenu pour sa simplicité d'installation, ses performances analytiques et sa bonne intégration à Python et dbt.

```

1 pipeline = dlt.pipeline(
2     pipeline_name="oulad_pipeline",
3     destination="duckdb",
4     dataset_name="raw",
5 )
6 pipeline.run([
7     student_info(), student_assessment(), student_vle(),
8     student_registration(), assessments(), courses(), vle(),
9 ])

```

Listing 3 - Principe de l'ingestion dlt

4. Transformation et validation avec dbt

dbt organise les transformations en trois niveaux. Les modèles de staging normalisent chaque table source. Les modèles intermédiaires agrègent l'engagement et les évaluations. Enfin, le mart `student_learning_features` réunit les variables nécessaires à l'analyse et les métadonnées utiles à l'interprétation.

- staging : renommage, typage et sélection des colonnes ;
- intermediate : calcul des interactions, jours actifs, soumissions et retards ;
- marts : jointure finale au niveau étudiant, module et présentation ;
- tests : contrôle des valeurs obligatoires et de la validité des scores.

```

1 cd dbt/educluster
2 dbt build

```

Listing 4 - Construction et contrôle des modèles dbt

5. Vue d'ensemble du flux de données

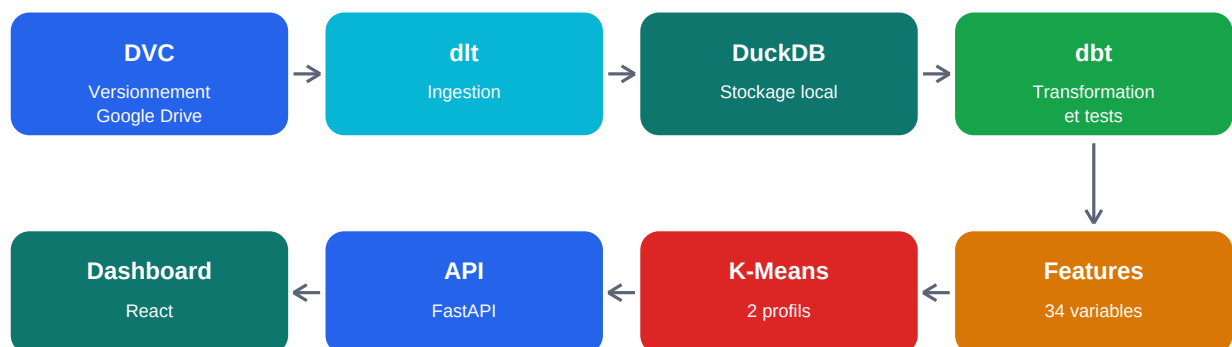


Figure 2 - Chaîne complète de la donnée jusqu'à l'interface

Chaque étape produit un artefact vérifiable pour l'étape suivante. Cette organisation réduit les dépendances implicites et permet d'identifier rapidement si une anomalie provient de la donnée, de la transformation, du modèle ou du service.

6. Granularité des observations et clés de rapprochement

La notion d'étudiant doit être interprétée avec précision dans cette application. Les agrégations SQL utilisent l'identifiant étudiant, le code du module et le code de la présentation. Une observation correspond donc à un étudiant dans un contexte de cours donné. Un même individu peut apparaître dans plusieurs contextes sans qu'il s'agisse nécessairement d'un doublon. Les 32 593 observations de référence ne doivent pas être présentées automatiquement comme autant de personnes distinctes. Cette distinction a des conséquences sur les tableaux de distribution, sur l'analyse des profils et sur toute tentative de généralisation des résultats à une population humaine.

Les clés de rapprochement assurent la cohérence entre les sources. Les soumissions sont associées aux évaluations par l'identifiant d'évaluation afin de retrouver les échéances et le contexte du module. Les interactions sont reliées aux ressources par l'identifiant de site pour connaître leur type. Les agrégats produits sont ensuite rattachés au profil de l'étudiant dans sa présentation de cours. Une mauvaise jointure pourrait multiplier les lignes, gonfler artificiellement les clics ou attribuer une évaluation au mauvais contexte. La validation des cardinalités et des clés est donc aussi importante que la vérification des valeurs numériques.

Le projet dispose de contrôles sur certaines colonnes et sur les scores, mais une amélioration utile serait d'ajouter des tests explicites sur l'unicité de la clé composée du mart. Il serait également pertinent de comparer les effectifs avant et après chaque jointure. Une différence attendue devrait être documentée ; une différence inexpliquée devrait interrompre la préparation. Ces tests permettraient de détecter les anomalies plus tôt que les simples erreurs de dimensions rencontrées au moment de l'entraînement. Ils représenteraient un complément au pipeline existant, et non une propriété déjà démontrée par les résultats de référence.

7. Différence entre stockage distant, cache et base analytique

Google Drive et DuckDB ne jouent pas le même rôle dans le système. Le premier est configuré comme destination distante de DVC pour conserver les objets de données versionnés. Le second est un fichier de base analytique local créé et utilisé par les traitements. Entre les deux, DVC met les données à disposition dans l'espace de travail et entretient un cache local. Les scripts d'ingestion lisent ensuite les CSV restaurés. Il existe donc plusieurs représentations du même patrimoine de données, chacune adaptée à un usage : conservation, récupération, transformation ou consommation par l'apprentissage.

La commande `dvc pull` ne se limite pas conceptuellement au dossier des données brutes. Elle peut récupérer les sorties suivies par les fichiers DVC et par le verrou du pipeline, sous réserve que les objets correspondants aient été publiés dans le stockage distant. Le fichier `dvc.yaml` déclare notamment le scaler et le modèle final comme sorties de calcul. Il ne faut donc pas déduire, à partir de la seule présence locale d'un fichier, la liste des objets réellement disponibles sur Google Drive. Cette disponibilité doit être vérifiée avec les outils DVC et les droits d'accès appropriés.

Google Drive constitue le stockage distant de DVC, tandis que DuckDB fournit la base analytique locale. DVC restaure les fichiers versionnés dans l'espace de travail ; dlt lit ensuite

les CSV et dbt transforme la base. La variable `EDUCLUSTER_DUCKDB_PATH` sélectionne le fichier DuckDB. La configuration Compose partage le même chemin entre les traitements, avec une exécution dbt limitée à un thread et 512 MB de mémoire.

8. Lecture des agrégations d'engagement

Les interactions sont d'abord regroupées par jour, étudiant, module et présentation. Cette étape permet de calculer les clics quotidiens et le nombre d'événements observés. Une seconde agrégation produit les jours actifs, les semaines actives, l'étendue entre la première et la dernière activité et plusieurs statistiques de volume. Le nombre de jours actifs compte uniquement les jours présents dans les traces. La moyenne quotidienne calculée sur ces lignes représente donc la moyenne des jours observés, et non nécessairement une moyenne sur chaque journée calendaire du cours. Cette définition doit être conservée dans les futures données d'entrée.

La variable `unique_resource_visits` mérite une attention particulière. Dans le SQL actuel, elle est construite comme la somme du nombre de ressources distinctes visitées chaque jour. Une même ressource consultée pendant plusieurs jours contribue plusieurs fois à cette somme. Il ne s'agit donc pas du nombre global de ressources uniques sur toute la période. De même, `avg_clicks_per_event` est obtenu par une moyenne des ratios quotidiens, ce qui peut différer du rapport entre les clics totaux et les événements totaux. Ces différences ne sont pas seulement terminologiques : elles changent la valeur numérique présentée au modèle.

La répartition par type d'activité complète ces indicateurs de volume. Le SQL distingue les contenus, les forums, les quiz ou questionnaires, les wikis et une catégorie résiduelle. Les ratios expriment ensuite chaque volume relativement au total des clics. Cette approche aide à séparer l'intensité globale de la nature des usages. Toutefois, le classement dépend de la nomenclature des ressources et du succès des jointures. Avant d'appliquer le pipeline à une autre plateforme, il faudrait vérifier que les catégories locales correspondent à celles attendues, faute de quoi les comparaisons entre profils perdraient une partie de leur sens.

9. Évaluations, retards et données manquantes

Les indicateurs d'évaluation rapprochent la date de soumission et la date prévue afin de construire un délai. Une valeur positive désigne une remise après l'échéance, une valeur négative une remise anticipée et zéro une remise à la date attendue. À partir de cette variable, le pipeline calcule des statistiques de retard et de ponctualité. Les scores moyens, minimaux et maximaux décrivent les évaluations effectivement observées. Ces informations enrichissent l'engagement mesuré par les clics, mais elles correspondent à un autre aspect du parcours et doivent être interprétées dans le contexte des modalités d'évaluation.

Le taux de soumission met en rapport le nombre d'évaluations distinctes remises et le nombre prévu dans la présentation du module. Ce dénominateur rend la mesure plus comparable qu'un simple nombre de remises, tout en supposant que la liste des évaluations attendues est correcte pour la période considérée. Dans une utilisation en cours de semestre, certaines échéances peuvent ne pas être encore arrivées. Employer sans adaptation le dénominateur de fin de cours pourrait alors faire apparaître comme faible une participation

parfaitement normale. La date d'observation constitue ainsi une information indispensable pour transformer cette analyse rétrospective en suivi précoce.

Le remplacement des valeurs manquantes par zéro apporte une solution calculatoire simple, mais rassemble parfois plusieurs situations. Une absence d'activité, une donnée non reçue et une statistique impossible à calculer peuvent aboutir à la même valeur. Le SQL utilise notamment COALESCE pour certaines dispersions ou certains délais. Cette convention rend la matrice utilisable, sans résoudre toutes les ambiguïtés métier. Une évolution possible consisterait à conserver des indicateurs de données manquantes ou à adopter des règles d'imputation distinctes selon la variable. Leur intérêt devrait être évalué dans une expérience documentée avant toute modification du modèle servi.

III - Préparation, entraînement et évaluation

1. Construction des caractéristiques

Le script `ml/prepare_data.py` lit le `mart.dbt` et sélectionne 34 caractéristiques comportementales. Elles couvrent l'intensité d'utilisation de la plateforme, la régularité, le type de ressources consultées et le comportement face aux évaluations.

Famille	Exemples de variables
Engagement général	<code>total_vle_interactions</code> , <code>total_clicks</code> , <code>active_days</code> , <code>active_weeks</code>
Régularité	<code>avg_daily_clicks</code> , <code>median_daily_clicks</code> , <code>max_daily_clicks</code> , <code>clicks_std</code>
Ressources	<code>clicks_content</code> , <code>clicks_forum</code> , <code>clicks_quiz</code> , <code>clicks_wiki</code>
Ratios d'usage	<code>content_ratio</code> , <code>forum_ratio</code> , <code>quiz_ratio</code> , <code>wiki_ratio</code> , <code>other_ratio</code>
Évaluations	<code>assessment_count</code> , <code>submission_rate</code> , <code>avg_assessment_score</code>
Ponctualité	<code>late_submission_rate</code> , <code>avg_submission_delay</code> , <code>on_time_submission_rate</code>

Tableau 3 - Familles de caractéristiques utilisées

2. Prétraitement

Les valeurs infinies sont remplacées, les données manquantes sont imputées à zéro et une transformation logarithmique `log1p` est appliquée aux variables de comptage très asymétriques. Un `StandardScaler` centre ensuite chaque variable autour de zéro et l'exprime sur une échelle comparable. Le scaler est sauvegardé pour reproduire exactement la transformation lors des prédictions futures.

```
1 for feature in LOG_FEATURES:
2     X[feature] = np.log1p(X[feature].clip(lower=0))
3
4 scaler = StandardScaler()
5 X_scaled = scaler.fit_transform(X)
6 joblib.dump(scaler, "data/processed/scaler.joblib")
```

Listing 5 - Log-transformation et standardisation

3. Comparaison des algorithmes

Le script de comparaison implémente K-Means, le clustering hiérarchique agglomératif et DBSCAN. GMM est présenté dans la documentation générale, mais n'est pas exécuté dans ce script. K-Means est le modèle final utilisé par l'API ; les résultats détaillés de tous les candidats ne sont pas disponibles pour établir ici un classement expérimental complet.

Algorithme	Principe	Intérêt / limite
K-Means	Répartition autour de centroïdes	Rapide et interprétable ; nécessite de fixer k
GMM	Mélange probabiliste de distributions	Appartenance souple ; hypothèses de distribution
Agglomératif	Fusion progressive de groupes	Lecture hiérarchique ; coût plus élevé
DBSCAN	Regroupement selon la densité	Détecte le bruit ; sensible aux paramètres

Tableau 4 - Méthodes de clustering considérées

4. Entraînement du modèle final

Le modèle final utilise deux clusters, une graine aléatoire fixée à 42 et dix initialisations. Le modèle apprend uniquement sur les 34 variables standardisées. Il est sérialisé au format Joblib dans `models/kmeans_final.joblib`.

```
1 model = KMeans(n_clusters=2, random_state=42, n_init=10)
2 labels = model.fit_predict(X)
3 joblib.dump(model, "models/kmeans_final.joblib")
```

Listing 6 - Entraînement et sauvegarde du modèle final

5. Évaluation et résultats

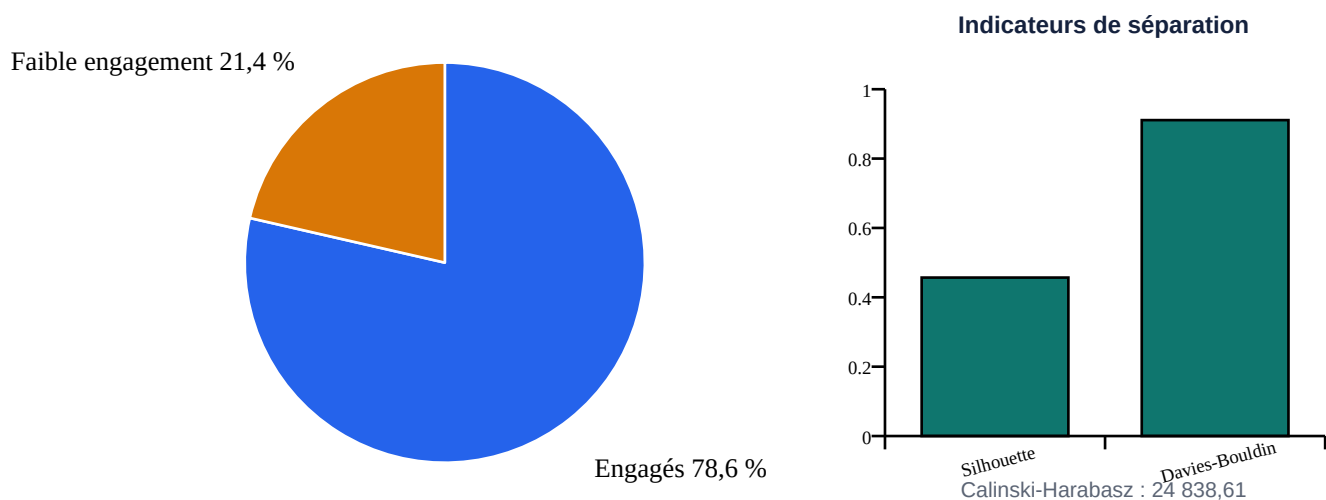


Figure 3 - Distribution des segments et principales métriques

Indicateur	Valeur de référence	Interprétation
Observations	32 593	Population analysée par le pipeline
Caractéristiques	34	Variables comportementales standardisées
Cluster 0	25 612 (78,6 %)	Profil actif / engagé
Cluster 1	6 981 (21,4 %)	Profil à faible engagement
Silhouette	0,4569	Séparation modérée et exploitable
Davies-Bouldin	0,9109	Compacité acceptable ; plus faible est meilleur
Calinski-Harabasz	24 838,61	Bonne séparation relative entre groupes

Tableau 5 - Résultats de référence du modèle K-Means

Les métriques évaluent la structure géométrique des groupes et non une exactitude de classification. En clustering non supervisé, il n'existe pas de vérité terrain fournie au modèle. L'interprétation doit donc combiner les scores, les profils moyens des groupes et l'expertise pédagogique.

6. Artefacts produits

Artefact	Utilité
clustering_features.csv	Matrice numérique utilisée pour l'entraînement
scaler.joblib	Transformation identique des futures observations
kmeans_final.joblib	Modèle final chargé par l'API
clustered_students.csv	Affectation de chaque observation à un cluster
cluster_profiles.csv	Résumé des profils par groupe
cluster_final_result_profile.csv	Analyse a posteriori du résultat final
kmeans_metrics.json	Paramètres, scores et effectifs des groupes

Tableau 6 - Sorties principales du pipeline ML

7. Effet des transformations sur les distances

K-Means compare les observations au moyen de distances dans un espace numérique. Sans préparation, une variable de clics dont les valeurs atteignent plusieurs milliers peut dominer une proportion comprise entre zéro et un. Le résultat traduirait alors principalement les unités choisies plutôt qu'un équilibre entre comportements. La transformation logarithmique réduit l'influence des écarts extrêmes, puis la standardisation exprime chaque variable par rapport à sa moyenne et à sa dispersion dans les données d'entraînement. Cette succession donne une place plus comparable aux dimensions, tout en conservant les différences relatives entre observations.

Un exemple pédagogique permet d'en comprendre l'effet. Passer de dix à cent clics et passer de cent à mille clics correspondent à des écarts absolus très différents, alors que les

deux changements représentent une multiplication par dix. Le logarithme rapproche cette lecture relative. La fonction `log1p` accepte en outre une valeur nulle, ce qui convient aux profils sans activité observée. Le code applique préalablement un bornage à zéro sur les variables concernées. Cette transformation est un choix de représentation, pas une preuve que toutes les variables ainsi traitées suivent une distribution idéale pour le modèle.

La normalisation introduit une dépendance au jeu d'entraînement. Les moyennes et écarts-types du scaler sont estimés une fois pendant la préparation, puis réutilisés à l'identique à l'inférence. Recalculer ces statistiques sur une seule requête supprimerait l'information utile et rendrait la comparaison avec les centres invalide. Le fichier `scaler.joblib` est donc un véritable artefact de modèle. Il doit accompagner la version de K-Means avec laquelle il a été produit, et toute modification de la liste des variables, de leur ordre ou des règles de transformation nécessite une réévaluation du lot complet.

8. Interprétation géométrique et choix de deux groupes

K-Means recherche des centres qui réduisent la dispersion des observations autour du centre de leur groupe. L'algorithme alterne l'affectation au centre le plus proche et le recalcul des centres à partir des observations affectées. Le processus conduit à une partition adaptée à la représentation numérique retenue. Deux caractéristiques fortement corrélées peuvent renforcer une même dimension comportementale, même après standardisation. Par exemple, le total de clics et plusieurs volumes par activité portent une information partiellement commune. L'interprétation des centres doit donc tenir compte de la redondance des variables et pas seulement du nombre de colonnes utilisées.

Le nombre de clusters est fixé à deux dans le script final. Ce choix donne une restitution facile à comprendre et correspond aux deux profils utilisés par l'application. Il ne démontre pas que la population ne comporte que deux façons d'apprendre. Une personne active dans les forums et une personne concentrée sur les contenus peuvent rejoindre le même grand groupe malgré des stratégies différentes. Une analyse complémentaire pourrait explorer plusieurs valeurs de k et étudier les compromis entre stabilité, séparation et lisibilité. Le rapport ne présente pas ce choix comme le résultat d'une optimisation exhaustive dont les essais ne sont pas fournis.

La graine fixée à 42 et les dix initialisations contribuent à rendre l'entraînement plus reproductible dans un environnement identique. Elles ne remplacent cependant pas le contrôle des versions logicielles, de l'ordre des observations et des données d'entrée. Les numéros des clusters ne possèdent pas de signification intrinsèque : leur interprétation découle d'une analyse des profils. L'application associe aujourd'hui le groupe zéro à l'engagement et le groupe un au faible engagement. Après un changement de données ou de méthode, il faudrait vérifier cette correspondance avant de conserver automatiquement les mêmes libellés dans l'API et l'interface.

9. Comparaison expérimentale et limites des preuves disponibles

Le script `compare_clustering.py` exécute effectivement K-Means, un clustering agglomératif avec liaison Ward et plusieurs configurations DBSCAN. Pour DBSCAN, le paramètre `eps` varie entre plusieurs valeurs, tandis que `min_samples` reste fixé à dix. Le script conserve les scores disponibles, le nombre de groupes, le nombre de points considérés comme bruit et la durée d'apprentissage. GMM est cité dans la présentation générale du projet, mais il n'est pas entraîné dans ce script de comparaison. Cette distinction évite d'attribuer au code des expérimentations dont la mise en œuvre n'est pas visible dans la version examinée.

La comparaison avec DBSCAN nécessite une lecture attentive des métriques. Le script exclut les points portant l'étiquette de bruit avant de calculer les indicateurs de séparation. Les scores peuvent donc porter sur une sous-population différente de celle évaluée pour K-Means. Une bonne silhouette sur un petit ensemble très homogène n'est pas directement équivalente à une silhouette obtenue en affectant toutes les observations. Le taux de bruit doit accompagner les métriques pour rendre la comparaison informative. Lorsque moins de deux groupes restent disponibles, le programme indique que les métriques ne peuvent pas être calculées au lieu d'inventer une valeur.

Les résultats numériques détaillés de tous les candidats ne sont pas présents parmi les fichiers de référence utilisés pour ce rapport. Il n'est donc pas possible d'affirmer qu'un algorithme est supérieur à tous les autres sur la base d'un tableau comparatif complet. Le constat vérifiable est que le service final charge un K-Means à deux groupes et que ses valeurs de référence sont fournies. Pour consolider le choix expérimental, une nouvelle campagne devrait publier les paramètres de chaque essai, la population évaluée, les scores, les temps et les artefacts associés, dans un environnement logiciel précisément identifié.

10. Lecture critique des métriques de référence

La silhouette de référence vaut 0,4569. Elle résume l'écart entre la proximité d'une observation avec son propre groupe et sa proximité avec le groupe concurrent. Une valeur moyenne positive traduit une séparation utile dans la représentation choisie, mais elle ne signifie pas que 45,69 % des étudiants sont correctement classés. Il ne s'agit pas d'une précision de classification. La moyenne peut également masquer des observations proches de la frontière ou des différences entre clusters. Une analyse plus détaillée examinerait la distribution des silhouettes individuelles et les profils dont l'affectation reste peu nette.

L'indice Davies-Bouldin de 0,9109 et l'indice Calinski-Harabasz de 24 838,61 complètent cette lecture en décrivant la compacité et la séparation des groupes selon d'autres formules. Ces valeurs sont surtout utiles pour comparer des partitions évaluées sur des données et des transformations compatibles. Elles ne constituent pas des seuils universels d'acceptation pédagogique. Une évolution de la taille du jeu, du nombre de variables ou du prétraitement peut modifier les scores. Le jugement de qualité doit donc associer les indicateurs numériques à l'examen des distributions, à la stabilité et à la pertinence des usages envisagés.

Les effectifs de référence totalisent 32 593 observations, dont 25 612 dans le premier cluster et 6 981 dans le second. Cette asymétrie n'est pas en soi un défaut : un clustering n'a pas l'obligation de produire des groupes de même taille. Elle invite néanmoins à étudier les

profils minoritaires et les observations atypiques. La part de faible engagement ne doit pas être lue comme un taux d'échec prédit. Le rapprochement avec `final_result` intervient après le clustering à des fins descriptives. Il peut aider à interpréter les groupes, mais ne démontre ni une causalité ni la capacité d'anticiper un résultat individuel.

11. Temporalité, fuites d'information et validation future

La préparation utilise des agrégats couvrant les données disponibles dans le `mart`. Elle n'introduit pas actuellement une date de coupure commune qui limiterait les observations aux premières semaines du cours. Les scores et les soumissions de la période entière peuvent donc contribuer à la segmentation. Le fait d'exclure `final_result` ne suffit pas à transformer cette expérience en dispositif de détection précoce. Pour soutenir une telle utilisation, il faudrait définir à quel moment la prédiction est produite et garantir que toutes les variables utilisées auraient effectivement été connues à cette date.

Une démarche de validation prospective pourrait construire des jeux à la fin de la deuxième, de la quatrième et de la sixième semaine. Chaque jeu utiliserait uniquement les interactions et évaluations disponibles jusque-là, avec des dénominateurs adaptés aux activités déjà attendues. Il serait ensuite possible d'étudier la stabilité des profils et leur évolution. Une telle démarche répondrait plus directement à la notion de parcours, car elle observerait des changements au cours du temps. Elle représente une extension méthodologique du projet actuel et nécessiterait de nouvelles transformations, de nouveaux artefacts et un protocole d'évaluation spécifique.

La validation devrait également tenir compte de la dépendance entre observations d'une même personne ou d'un même cours. Un découpage aléatoire naïf pourrait mélanger des contextes très proches entre développement et évaluation. Une séparation par présentation ou par période rendrait l'analyse de généralisation plus exigeante. Dans un cadre non supervisé, il serait utile de mesurer la stabilité des centres, la répartition des affectations et la cohérence des profils sur les nouvelles données. L'objectif serait de vérifier que les groupes restent interprétables au-delà du jeu utilisé pour les construire, sans réduire la validation à un seul score.

IV - Traçabilité et orchestration MLOps

1. Suivi des expériences avec MLflow

MLflow enregistre le nom de l'expérience, les hyperparamètres, la taille du jeu de données, les métriques et le modèle. L'expérience EduCluster - KMeans rend les essais comparables et facilite l'audit de la version retenue.

```
1 mlflow.set_experiment("EduCluster-KMeans")
2 with mlflow.start_run(run_name="kmeans-final-k2"):
3     mlflow.log_params({"n_clusters": 2, "random_state": 42, "n_init": 10})
4     mlflow.log_metrics({
5         "silhouette_score": silhouette,
6         "davies_bouldin_score": davies_bouldin,
7         "calinski_harabasz_score": calinski_harabasz,
8     })
9     mlflow.sklearn.log_model(model, "kmeans_model")
```

Listing 7 - Journalisation de l'expérience MLflow

2. Reproductibilité avec DVC

Le fichier `dvc.yaml` relie explicitement les dépendances et sorties des étapes de préparation et d'entraînement. DVC recalcule une étape lorsque son code ou l'une de ses entrées change. Le fichier `dvc.lock` conserve les empreintes de l'exécution reproductible.

```
1 stages:
2   prepare_features:
3     cmd: python ml/prepare_data.py
4     outs:
5       - data/processed/clustering_features.csv
6       - data/processed/scaler.joblib
7   train_kmeans:
8     cmd: python ml/clustering.py
9     outs:
10      - models/kmeans_final.joblib
11     metrics:
12      - data/processed/kmeans_metrics.json
```

Listing 8 - Résumé du pipeline DVC

3. Orchestration avec Dagster

Dagster représente chaque sortie importante comme un actif : données brutes OULAD, mart étudiant, jeu de caractéristiques, modèle K-Means et expérience MLflow. Les dépendances garantissent l'ordre d'exécution et rendent l'état du pipeline observable.

Actif Dagster	Dépendance	Sortie principale
oulad_raw	Données DVC locales	Base DuckDB, schéma raw
student_learning_features	oulad_raw	Mart dbt
clustering_features	student_learning_features	CSV standardisé + scaler
kmeans_model	clustering_features	Modèle, métriques et profils
mlflow_kmeans_experiment	kmeans_model	Run MLflow et artefacts

Tableau 7 - Graphe d'actifs Dagster

4. Cycle de vie du modèle

Dans la version actuelle, le déploiement utilise le modèle et le scaler déjà présents dans l'espace de travail. Le pipeline CI/CD ne déclenche pas automatiquement un `dvc pull` ni un nouvel entraînement. Cette séparation évite qu'une modification d'interface ou d'API entraîne involontairement un changement de modèle.

5. Versionnement du modèle et contrat des artefacts

Le fichier `models/model_manifest.json` décrit le modèle livré : identifiant de version, révision Git d'entraînement, empreintes SHA-256 du modèle et du scaler, ordre des caractéristiques et liste des variables soumises à `log1p`. Il associe également les numéros de clusters à leurs profils. Le script `ml/clustering.py` produit ce manifeste avec les artefacts d'entraînement. L'ensemble forme le contrat entre préparation, apprentissage et inférence.

Le validateur `scripts/validate_model_bundle.py` vérifie la présence des fichiers, leurs empreintes et la compatibilité des variables. FastAPI charge cet ensemble au démarrage et compare le schéma de ses 34 champs à l'ordre déclaré. Les routes `/health` et `/model-info` exposent la version et l'empreinte du modèle chargé. Git conserve le code, DVC décrit les dépendances et sorties versionnées, et MLflow enregistre les informations de l'expérience. Ces mécanismes rendent la version servie identifiable sans imposer de réentraînement à chaque démarrage.

6. Journalisation du modèle existant dans MLflow

L'actif MLflow appelle `scripts/log_to_mlflow.py`. Ce script valide le bundle puis charge le modèle déjà sérialisé. Il sélectionne la destination de suivi à partir d'un argument, de `MLFLOW_TRACKING_URI` ou d'un répertoire `mlruns` local. Dans Compose, les traitements utilisent `http://mlflow:5000` sur le réseau interne. L'expérience `EduCluster-KMeans` conserve le modèle, le scaler, le manifeste et le fichier d'évaluation annexé.

Le script recherche d'abord un run portant la même empreinte `model_sha256`, afin de ne pas enregistrer plusieurs fois le même modèle. Les paramètres et les tags précisent la version, la révision d'entraînement et la validation du bundle. La silhouette, Davies-Bouldin et Calinski-Harabasz journalisés par ce script proviennent de `api/reference_results.json` ; l'inertie provient du modèle chargé. Cette provenance définit le périmètre des valeurs de référence et

ne constitue pas, à elle seule, une nouvelle évaluation sur d'autres données.

Le service MLflow démarre via `scripts/start_mlflow.py`, attend la disponibilité du serveur, puis journalise le bundle déployé. Son stockage SQLite et les artefacts résident dans `mlflow-data`, monté sur `/mlflow-store`. Les expériences persistent indépendamment de la durée de vie du conteneur. Une consultation de MLflow et une matérialisation Dagster sont donc deux opérations distinctes.

7. Matérialisation, tests et métadonnées Dagster

Le graphe ordonne cinq actifs : `oulad_raw`, `student_learning_features`, `clustering_features`, `kmeans_model` et `mlflow_kmeans_experiment`. Une matérialisation demande la production effective des actifs sélectionnés. L'interface Assets présente les dépendances et l'interface Runs donne accès aux événements et aux journaux de chaque exécution. Les commandes externes utilisent `check=True` pour conditionner la réussite de l'actif à celle du processus.

L'actif `student_learning_features` exécute `dbt build` avec les chemins explicites du projet et des profils. Les transformations et leurs tests font ainsi partie de l'orchestration. La préparation mesure les dimensions réelles du CSV, tandis que l'actif K-Means lit `kmeans_metrics.json` pour exposer les effectifs, le nombre de variables et les scores produits. Les métadonnées décrivent donc les sorties de l'exécution, plutôt que des constantes de présentation.

8. Séparation du serveur et du worker

Le service dagster publie l'interface Web sur le port 4204. Le service dagster-worker exécute dagster-daemon et les traitements, sans port public. Les deux partagent le stockage d'instance dagster-data, le répertoire de travail dagster-workspace et les données brutes. La file `QueuedRunCoordinator` limite les exécutions simultanées à un run.

La configuration attribue au serveur Web 0,5 CPU et 512 MB. Le worker dispose par défaut de 1 CPU et 1 536 MB, paramétrables par `TRAINING_CPUS` et `TRAINING_MEMORY`. Les bibliothèques numériques et les étapes dlt utilisent un parallélisme borné. Cette séparation répartit explicitement les ressources entre orchestration, calcul et services de consultation.

9. Publication contrôlée du modèle

La matérialisation génère les sorties d'apprentissage. Leur publication dans l'API reste une étape distincte : sélectionner un ensemble cohérent, le valider, reconstruire l'image API, puis recréer le service pour charger le modèle livré. La réussite d'un run n'effectue pas automatiquement cette promotion. La configuration CI/CD ne déclenche pas non plus de `dvc pull` ni d'entraînement à chaque push applicatif.

Un registre avec validation et promotion contrôlée constitue une évolution possible. Le retour à une version antérieure doit conserver l'association du modèle, du scaler, du manifeste et des résultats descriptifs. La surveillance des distributions et le réentraînement sur décision humaine prolongent ce cycle sans modifier le sens pédagogique des profils.

V - API, dashboard et conteneurisation

1. API FastAPI

Au démarrage, l'API charge le modèle K-Means et le scaler. Elle expose des points d'accès simples pour vérifier l'état du service, consulter les statistiques, lire les métriques du modèle et affecter un nouveau profil à un cluster.

Méthode	Route	Fonction
GET	/	Identité et version du service
GET	/health	État de santé et modèle chargé
GET	/statistics	Effectifs et pourcentages des clusters
GET	/model-info	Algorithme, nombre de variables et métriques
POST	/predict	Prétraitement puis affectation d'un nouveau profil

Tableau 8 - Contrat fonctionnel de l'API

1.1. Séquence d'une prédiction

- validation des 34 valeurs reçues avec Pydantic ;
- application de log1p aux variables de comptage ;
- standardisation avec scaler.joblib ;
- appel à model.predict ;
- retour du numéro de cluster et du profil lisible.

```
1 POST /predict
2 {
3   "total_clicks": 620,
4   "active_days": 45,
5   "submission_rate": 0.85,
6   "avg_assessment_score": 72,
7   "...": "30 autres caractéristiques"
8 }
9
10 Réponse : {"cluster": 0, "profile": "Active / Engaged Learner"}
```

Listing 9 - Format illustratif, requête abrégée et réponse possible

2. Dashboard React

Le tableau de bord React consomme l'API sans exécuter le pipeline ML. La page principale présente la population, la distribution des segments, leurs descriptions et les métriques. La page de prédiction permet de saisir un nouveau comportement étudiant.

- cartes synthétiques : total, apprenants engagés, faible engagement et nombre de clusters ;
- visualisation de la distribution des deux groupes ;
- description pédagogique des profils ;

- affichage des métriques de qualité ;
- formulaire complet de prédiction et retour du profil.

3. Conteneurisation Docker

Cinq services composent le déploiement : API FastAPI, MLflow, dashboard React servi par Nginx, serveur Web Dagster et worker Dagster. Les quatre interfaces disposent de ports distincts. Le worker exécute le daemon et les traitements sans publier de port. Docker Compose définit les dépendances de santé, les volumes persistants et les ressources attribuées aux traitements.

Service	Port public	Port interne	Rôle
api	4201	8000	API FastAPI et modèle
mlflow	4202	5000	Consultation des expériences
dashboard	4203	80	Interface Web React/Nginx
dagster	4204	3000	Interface et graphe des actifs
dagster-worker	Aucun	Aucun	Daemon et exécution des traitements

Tableau 9 - Services Docker du projet

```

1 docker compose config --quiet
2 docker compose build api dashboard mlflow dagster
3 docker compose up -d
4 docker compose ps
5 docker compose down

```

Listing 10 - Gestion de la pile Docker

4. Contrat des requêtes et validation des valeurs

Le schéma StudentFeatures impose la présence de 34 champs numériques. Une requête incomplète ou contenant une chaîne non convertible est rejetée par la validation avant l'appel au modèle. Cette protection évite plusieurs erreurs de format, mais elle ne garantit pas encore la cohérence pédagogique des valeurs. Dans le code actuel, les champs sont principalement déclarés comme nombres flottants sans contraintes détaillées de bornes. Des contrôles supplémentaires seraient utiles pour distinguer une proportion d'un pourcentage, imposer un nombre de jours plausible ou vérifier qu'un minimum de scores ne dépasse pas le maximum.

La préparation de la requête respecte une liste ordonnée de noms de variables. Cet ordre est déterminant, car le tableau numérique transmis au scaler ne contient pas la signification textuelle de chaque position. Inverser deux champs peut produire une affectation sans provoquer d'erreur technique, tout en modifiant profondément le sens du profil. Le test de régression existant reconstruit cette transformation et compare le résultat à celui du modèle réel. Une évolution plus robuste pourrait centraliser la liste et les règles dans un module partagé afin de limiter le risque de divergence entre préparation d'entraînement et service d'inférence.

Le bornage des valeurs négatives à zéro sur les variables transformées mérite d'être documenté auprès des utilisateurs. Il est naturel pour un comptage, mais les délais de soumission peuvent être négatifs lorsqu'un travail est rendu en avance. Le code applique actuellement cette règle à plusieurs statistiques de délai. La préparation perd donc une partie de la distinction entre les différents niveaux d'avance. Cette observation ne justifie pas une modification isolée de l'API : changer cette règle nécessiterait de reconstruire la préparation, de réentraîner et de réévaluer le modèle pour maintenir la cohérence de l'ensemble.

5. Provenance des statistiques et comportement en cas d'absence

Le tableau de bord ne dépend pas uniquement des prédictions individuelles. Il consulte aussi des effectifs et des scores agrégés. Le module `api/dashboard.py` recherche d'abord les résultats locaux du pipeline, puis utilise les données de référence lorsque les fichiers correspondants sont absents et que les conditions prévues sont remplies. Le fichier `api/reference_results.json` contient les effectifs et les métriques livrés avec le projet. Ce mécanisme permet une démonstration de l'interface sans régénérer l'ensemble du pipeline, tout en conservant un champ source qui précise l'origine de la réponse.

Cette provenance doit être visible et comprise. Un utilisateur pourrait sinon penser que chaque ouverture de la page déclenche une nouvelle analyse ou que les statistiques décrivent immédiatement un fichier qu'il vient de remplacer. L'interface comporte un message signalant l'utilisation des résultats de référence. Les fonctions de lecture contrôlent également la cohérence des effectifs et la finitude des métriques. Si un fichier local incohérent est présent, certaines routes retournent une indisponibilité plutôt que de masquer silencieusement l'anomalie. Ce comportement contribue à rendre les erreurs de production visibles au lieu de présenter des nombres plausibles mais contradictoires.

Les descriptions qualitatives des profils sont intégrées au frontend, alors que les effectifs et les scores proviennent de l'API. Après un réentraînement, il faudrait vérifier à la fois les nombres et les textes. Une inversion des numéros de clusters ou l'apparition de profils différents pourrait rendre les commentaires existants trompeurs même si les endpoints continuent de fonctionner. Une amélioration possible serait de versionner les descriptions avec le modèle ou de les servir depuis une source commune. Le projet met ainsi en évidence une dimension souvent négligée de la maintenance : le contenu pédagogique de l'interface dépend lui aussi du modèle déployé.

6. Communication et configuration des interfaces

Le navigateur consulte le dashboard sur le port 4203, puis appelle FastAPI sur 4201. MLflow et Dagster sont accessibles respectivement sur 4202 et 4204. Les variables `VITE_API_URL`, `VITE_MLFLOW_URL` et `VITE_DAGSTER_URL` sont intégrées à la construction du frontend. `CORS_ORIGINS` déclare les origines autorisées à utiliser l'API. En local, `.env.local` fournit les adresses localhost adaptées à ces quatre interfaces.

Les conteneurs communiquent par les noms des services sur le réseau Compose. Le worker utilise notamment `mlflow:5000`, tandis que le navigateur utilise le port publié 4202. Le dashboard dépend d'une API saine, et les composants concernés attendent la disponibilité de

MLflow. Les sondes de santé et les ports internes ont des rôles distincts des URL visibles par l'utilisateur.

7. Persistance et cycle de chargement

Le modèle, le scaler et le manifeste sont copiés dans l'image API au moment de sa construction. FastAPI les valide puis les charge au démarrage. Les requêtes réutilisent ces objets en mémoire. Modifier un fichier de travail n'actualise donc pas automatiquement le modèle d'un processus déjà démarré ; la publication suit le cycle de construction et de déploiement prévu.

Le stockage des expériences MLflow est conservé dans `mlflow-data`. L'historique de l'instance Dagster réside dans `dagster-data`, et sa base analytique de travail se trouve dans `dagster-workspace`. Les CSV OULAD sont montés en lecture seule depuis `data/raw`. Cette organisation explicite quelles données persistent et quels artefacts appartiennent aux images applicatives.

VI - Intégration et déploiement continu

1. Architecture CI/CD

Le workflow GitHub Actions s'exécute sur les demandes de fusion et les pushes vers la branche principale. Les contrôles sont répartis en quatre jobs indépendants : API et modèle persisté, sécurité des ressources d'entraînement, frontend et images Docker. Le déploiement Komodo dépend de la réussite de ces quatre contrôles.

Job	Contrôles réalisés	Condition
api	Validation du bundle, régressions API et tests du webhook	Pull request et push
training-safety	Tests de configuration des ressources d'entraînement	Pull request et push
frontend	Installation, tests Vitest et build Vite	Pull request et push
images	Validation Compose et construction des quatre images	Pull request et push
deploy	Webhook signé puis vérifications HTTP de la stack	Push sur main après succès

Tableau 10 - Jobs de la chaîne CI/CD

2. Validation de l'API et du modèle persisté

Les tests chargent le modèle et le scaler livrés avec l'application, vérifient les routes du dashboard et contrôlent que les réponses restent cohérentes. Cette approche détecte une incompatibilité de version scikit-learn, un artefact manquant ou une régression du contrat de l'API avant la construction des images.

3. Validation du frontend

Le job frontend installe les dépendances avec `npm ci`, exécute les tests avec Vitest et génère le build de production. Les composants de formulaire et le service de communication avec l'API sont ainsi vérifiés avant déploiement.

4. Déploiement Komodo

Après validation de la CI, un script Python envoie une requête signée au webhook Komodo. Deux secrets GitHub sont requis : `KOMODO_WEBHOOK_URL` et `KOMODO_WEBHOOK_SECRET`. Komodo met ensuite à jour la pile Docker déjà configurée sur la machine cible.

```
1 push sur main
2   -> API + training-safety + frontend + images Docker
3   -> vérification du commit le plus récent
4   -> webhook signé
5   -> déploiement de la stack Komodo
```

Figure 4 - Enchaînement logique de la livraison continue

5. Garanties apportées

- aucun déploiement si un test ou une image échoue ;
- pas de déploiement d'un commit devenu obsolète ;
- webhook authentifié par secret ;
- même configuration Docker entre validation et environnement cible ;
- séparation claire entre réentraînement du modèle et déploiement applicatif.

6. Les quatre contrôles avant livraison

Le job api valide le bundle puis exécute les tests de régression de l'API et du webhook. Le job training-safety vérifie la configuration de sécurité des ressources d'entraînement. Le job frontend installe les dépendances, lance les tests puis construit React. Le job images contrôle Compose et construit les images API, dashboard, MLflow et Dagster. Le worker réutilise l'image Dagster, ce qui donne quatre images pour cinq services.

Ces jobs contrôlent le code, les artefacts et la construction. Ils ne réalisent pas un nouvel entraînement complet. Le job deploy dépend de leur réussite, exige un push sur main et vérifie que le commit est toujours le plus récent. Le script déclenche ensuite le webhook signé vers la stack Komodo configurée.

7. Vérification après déploiement

Après l'envoi du webhook, le workflow interroge l'API sur /health, le dashboard sur sa racine et Dagster sur /server_info. La vérification API recherche également model_version dans la réponse. Une boucle bornée attend la disponibilité des trois interfaces avant de déclarer la livraison saine. Le contrôle configuré ne couvre pas directement l'interface publique MLflow.

La construction des images dans GitHub Actions valide leur capacité à être produites. Le workflow décrit ici déclenche ensuite la mise à jour de la stack existante dans Komodo ; il ne déclare pas de publication d'images dans un registre externe. Le suivi des expériences, l'orchestration d'apprentissage et la livraison applicative restent des responsabilités complémentaires.

VII - Reproduction du projet

1. Prérequis

Outil	Usage
Python 3.11 ou 3.12	Pipeline, API, tests et orchestration
Node.js 22.12+ et npm	Dashboard React
Docker + Docker Compose	Exécution conteneurisée
DVC avec support Google Drive	Téléchargement des données OULAD
dbt-duckdb	Transformation et tests SQL
Accès au remote DVC	Authentification Google Drive

Tableau 11 - Environnement requis

2. Installation locale

```
1 py -3.12 -m venv .env-dash-board
2 .\.env-dash-board\Scripts\python.exe -m pip install -r requirements.txt
3
4 cd dashboard
5 npm ci
6 cd ..
```

Listing 11 - Installation des dépendances

3. Reproduction complète du pipeline de données et ML

```
1 dvc pull
2 python dlt_pipeline/load_oulad.py
3 dbt build --project-dir dbt/educcluster --profiles-dir dbt
4 python ml/prepare_data.py
5 python ml/clustering.py
6 python scripts/validate_model_bundle.py
7 python scripts/log_to_mlflow.py
```

Listing 12 - Exécution complète du pipeline

Après ces étapes, la base DuckDB, les données préparées, le scaler, le modèle, les métriques et les profils de clusters sont disponibles localement. L'interface MLflow peut être ouverte pour consulter le run enregistré.

4. Lancement local des services

```
1 # Depuis la racine du projet, avec Docker Desktop actif
2 docker compose --env-file .env.local up -d --build
3 docker compose --env-file .env.local ps
4
5 # Chaque interface locale dispose de son propre port
6 # API 4201, MLflow 4202, dashboard 4203, Dagster 4204
```

Listing 13 - Lancement local des quatre interfaces

5. Lancement simplifié avec Docker

```
1 # Exemple de contenu de .env.local
2 VITE_API_URL=http://localhost:4201
3 VITE_MLFLOW_URL=http://localhost:4202
4 VITE_DAGSTER_URL=http://localhost:4204
5 CORS_ORIGINS=http://localhost:4203,http://127.0.0.1:4203
6
7 docker compose --env-file .env.local up -d --build
```

Listing 14 - Configuration locale du frontend et de CORS

En configuration locale : dashboard sur <http://localhost:4203>, API sur <http://localhost:4201/docs>, MLflow sur <http://localhost:4202> et Dagster sur <http://localhost:4204>. Les URL de navigateur se configurent avant la construction du frontend.

6. Organisation des données et des chemins

Les sept CSV doivent être disponibles dans `data/raw/oulad` : `studentInfo.csv`, `studentRegistration.csv`, `studentAssessment.csv`, `studentVle.csv`, `assessments.csv`, `courses.csv` et `vle.csv`. DVC permet de restaurer les données versionnées avec les droits du remote configuré. `OULAD_DATA_DIR` désigne ce répertoire pour l'ingestion.

En Compose, les CSV apparaissent sous `/app/data/raw/oulad`. `EDUCLUSTER_DUCKDB_PATH` et `DESTINATION_DUCKDB_CREDENTIALS` désignent `/app/runtime/oulad_pipeline.duckdb`. Les traitements utilisent ainsi une base commune. Le profil dbt lit le chemin depuis l'environnement, ce qui rend la configuration indépendante d'un chemin absolu propre à un poste.

7. Exécution depuis Dagster

Après le démarrage des services, l'utilisateur ouvre <http://localhost:4204>, consulte Assets, sélectionne les cinq actifs puis lance Materialize selected. La page Runs permet de suivre chaque étape et les métadonnées associées. Les données sources sont un prérequis de cette exécution. Le graphique de dépendances décrit le workflow configuré, tandis que les événements de matérialisation décrivent les exécutions enregistrées.

L'application peut également fonctionner avec les artefacts livrés sans relancer ce pipeline. Le dashboard et FastAPI servent alors la segmentation disponible. Pour une nouvelle version du modèle, la préparation, l'entraînement, l'évaluation et la publication doivent suivre le cycle documenté. Les résultats chiffrés du rapport demeurent les valeurs de référence du projet.

VIII - Validation, limites et perspectives

1. Validation du fonctionnement

La validation porte sur les transformations dbt, la cohérence du bundle, les tests API et frontend, la construction Docker et les sondes de disponibilité après déploiement. Les métriques de référence documentent la segmentation livrée. Leur présence ne remplace pas une évaluation de généralisation sur une autre population.

2. Limites actuelles

- deux clusters simplifient une réalité pédagogique potentiellement plus nuancée ;
- les résultats dépendent du contexte OULAD et ne sont pas automatiquement transférables à une autre institution ;
- l'étiquette « à risque » nécessite une validation métier et un usage prudent ;
- la dérive des comportements et de la distribution des données n'est pas encore surveillée automatiquement ;
- le réentraînement n'est pas intégré au pipeline de déploiement applicatif ;
- le modèle, le scaler et leur manifeste forment un ensemble validé avant la construction de l'image API.

3. Perspectives d'amélioration

- ajouter un suivi de dérive des données et des distributions de clusters ;
- mettre en place un registre de modèles avec validation et promotion contrôlée ;
- étudier plusieurs nombres de clusters et compléter l'évaluation par une expertise pédagogique ;
- automatiser un pipeline de réentraînement distinct avec approbation humaine ;
- enrichir l'observabilité : journaux, métriques de service et alertes ;
- documenter la gouvernance, la confidentialité et les conditions d'usage des profils.

4. Apports pédagogiques du projet

Ce projet illustre la différence entre un prototype de machine learning et un produit MLOps. L'algorithme ne constitue qu'une partie de la solution. La valeur du système repose aussi sur la qualité des données, la reproductibilité, la traçabilité, les tests, la stabilité des interfaces et la capacité de déployer sans rompre le service.

6. Interprétation pédagogique et limites des données

L'engagement numérique est un indicateur partiel du travail d'apprentissage. Une personne peut télécharger un document puis l'étudier hors ligne, travailler à partir d'un manuel ou participer à des échanges extérieurs à la plateforme. À l'inverse, un grand nombre de clics peut traduire une navigation hésitante plutôt qu'une compréhension approfondie. Les variables disponibles décrivent donc des traces d'usage et certains résultats d'évaluation, pas l'ensemble de l'activité cognitive. Les libellés de clusters doivent préserver cette distinction afin de soutenir une discussion pédagogique plutôt que d'enfermer les étudiants dans une catégorie définitive.

L'exclusion des variables démographiques de l'entraînement réduit certaines dépendances explicites, mais ne démontre pas l'absence de biais. Des variables comportementales peuvent être associées à des contraintes d'accès, de disponibilité ou de familiarité avec les outils numériques. Une évaluation responsable examinerait les erreurs d'interprétation possibles et les différences de représentation entre groupes, selon un protocole adapté au contexte. Ce rapport ne revendique pas qu'une telle étude a été réalisée. Il souligne que la qualité d'une solution éducative ne se résume pas à la séparation géométrique des clusters et qu'un usage réel requiert un dialogue avec les acteurs concernés.

La comparaison avec `final_result` peut être informative sans transformer les groupes en catégories causales. Une association entre faible engagement et résultat défavorable peut refléter plusieurs mécanismes simultanés. L'application ne permet pas d'établir que modifier un nombre de clics provoquerait une amélioration du résultat. Une action d'accompagnement devrait être fondée sur un besoin confirmé, puis son effet étudié avec un protocole distinct. Cette prudence d'interprétation protège la cohérence scientifique du projet et permet de présenter sa valeur réelle : organiser les observations et rendre certains profils plus visibles pour une analyse humaine.

7. Surveillance future et évolution des distributions

Le système actuel fournit des métriques de clustering et des endpoints de consultation, mais ne met pas en place une surveillance automatique de dérive. Une évolution des cours, des ressources ou des pratiques pourrait modifier les valeurs reçues par le modèle. Les centres resteraient alors ceux de l'entraînement initial, même si la population a changé. Une première amélioration serait de suivre les distributions des principales caractéristiques, le nombre de requêtes et la proportion des affectations par période. Ces observations aideraient à identifier un changement durable avant de décider si une investigation ou un réentraînement est nécessaire.

La surveillance du service doit être distinguée de celle du modèle. Une augmentation des erreurs HTTP ou des temps de réponse relève de la disponibilité technique. Une modification des distributions de variables ou des distances aux centres relève de la pertinence de la représentation. Un service peut répondre rapidement avec un modèle devenu moins adapté, ou disposer d'un modèle pertinent tout en rencontrant des problèmes réseau. Les deux dimensions nécessitent des indicateurs et des responsabilités distincts. Le projet peut évoluer vers cette double observation en commençant par des mesures simples, documentées et associées à des

actions concrètes.

Les seuils d'alerte ne devraient pas être choisis arbitrairement. Une nouvelle session de cours peut légitimement présenter une activité plus faible pendant ses premières semaines. Une hausse d'un ratio peut résulter d'un changement de structure des ressources plutôt que d'un comportement inhabituel. Les références de surveillance doivent donc intégrer le calendrier et le contexte. Une alerte utile signale un écart interprétable et propose une vérification, plutôt qu'un jugement automatique. Cette perspective prolonge l'objectif pédagogique d'EduCluster en reliant la maintenance du modèle à l'évolution réelle des conditions d'apprentissage.

8. Perspectives de consolidation

Le projet dispose désormais d'un manifeste versionné, d'une validation de bundle, de tests dbt intégrés à Dagster et de métadonnées issues des sorties. La suite du travail peut porter sur une campagne comparative complète, un registre avec promotion contrôlée et une validation temporelle des profils. Les contrôles de santé applicative complètent ces mécanismes sans remplacer l'évaluation scientifique du clustering.

Le suivi automatique de dérive et le réentraînement déclenché sur alerte restent des perspectives. Une extension pédagogique pourrait étudier les transitions de profils dans le temps et l'effet d'actions d'accompagnement, avec les enseignants. Ces travaux demandent un protocole d'évaluation distinct des métriques de séparation géométrique.

Conclusion générale

EduCluster répond à la problématique d'analyse des parcours d'apprentissage en proposant une segmentation comportementale de la population étudiante. À partir des traces OULAD, la chaîne transforme des fichiers bruts en indicateurs structurés, prépare 34 variables, entraîne un K-Means à deux groupes et rend les résultats accessibles dans une application Web.

La dimension MLOps est centrale : DVC relie les résultats à une version de données, dbt contrôle les transformations, MLflow documente l'expérience, Dagster explicite les dépendances, Docker normalise l'exécution et GitHub Actions protège la livraison par des tests automatisés avant le déploiement Komodo.

Le système obtenu constitue une base cohérente pour l'aide au suivi pédagogique. Son usage doit rester explicable, supervisé et orienté vers l'accompagnement. Les profils détectés sont des signaux d'analyse, non des décisions sur les étudiants.

Bilan méthodologique et perspectives de transfert

L'apport principal du projet réside dans la mise en relation de plusieurs niveaux de travail habituellement présentés séparément. L'ingénierie des données prépare des observations exploitables, la modélisation produit une segmentation, et l'ingénierie logicielle rend cette segmentation accessible. La démarche MLOps relie ces activités par des dépendances, des contrôles et des artefacts. Le résultat est une architecture dont le fonctionnement peut être expliqué depuis les fichiers sources jusqu'à la réponse envoyée au navigateur. Cette compréhension de bout en bout constitue un acquis important dans une formation en intelligence artificielle orientée vers la réalisation de systèmes utilisables.

Les limites identifiées ne diminuent pas cet intérêt pédagogique ; elles indiquent les étapes nécessaires pour passer d'une démonstration reproductible à un service institutionnel. La cohérence des configurations, l'identification des versions, la validation temporelle et l'interprétation des profils constituent des conditions de confiance. Le rapport distingue ces besoins des fonctionnalités déjà observées, afin de fournir une base de travail honnête et utile. Une équipe qui reprend le projet peut ainsi choisir ses priorités en fonction du contexte, sans supposer que tous les mécanismes de gouvernance ou de surveillance sont déjà opérationnels.

Le transfert vers un autre établissement demanderait enfin de reconstruire le lien entre les traces locales et les définitions des indicateurs. Les ressources, les calendriers, les modalités d'évaluation et les pratiques pédagogiques peuvent différer de celles d'OULAD. Il faudrait adapter les transformations, vérifier les distributions et réexaminer les profils avec les acteurs concernés. L'architecture technique peut être réutilisée, mais le sens du modèle doit être réévalué. Cette conclusion rappelle le principe directeur du projet : une chaîne MLOps fiable doit rendre les calculs reproductibles tout en maintenant une interprétation contextualisée des données d'apprentissage.

Références techniques

- Open University Learning Analytics Dataset (OULAD).
- Documentation DVC - versionnement des données et pipelines reproductibles.
- Documentation dlt - ingestion de données vers DuckDB.
- Documentation dbt - transformations SQL et tests de qualité.
- Documentation scikit-learn - KMeans et métriques de clustering.
- Documentation MLflow - suivi des expériences et artefacts.
- Documentation FastAPI, Docker Compose, GitHub Actions et Komodo.